

cs2521

Week 1 | Recap

Ronnie Low

Table of Contents

Table of Contents	1
Agenda	5
Week 1 Agenda	6
the elephant	7
The elephant	8
Introductions	9
whoami	10
Your introductions	11
Revision	12
Pointers	13
Malloc/Heap	14
Examples	15

Stack vs Heap example	15
Answer:	16
Swap example	17
Answer	18
Heap allocation example	19
Answer	20
Answer	22
Linked Lists	23
Answer:	25
Answer	27
Answer	29
Answer	31

Answer	37
--------------	----

Agenda

Week 1 Agenda

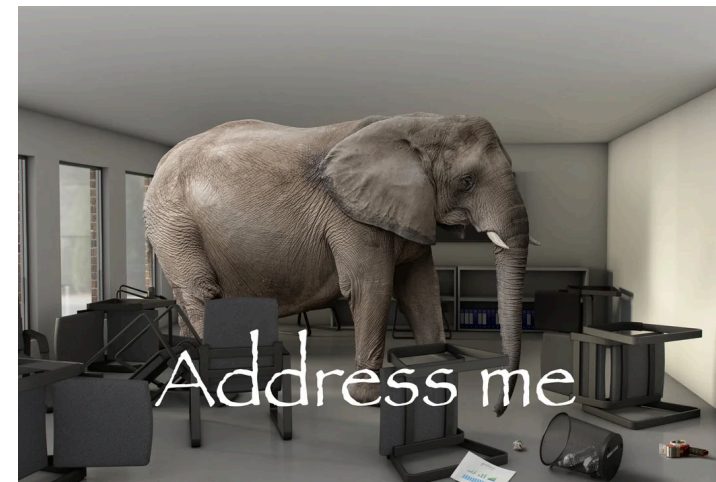
- The elephant
- Introductions
- Revision (cs1511)
 - Pointers
 - Malloc/heap memory
 - Linked lists

the elephant

The elephant

the elephant

- AI isn't allowed in 2521
- There is an exam hurdle
- 2521 will come in handy for job interviews (leetcode)



- Would you rather:
 - Spend an extra hour a week on labs and go into the exam confident
 - Cram multiple all nighters few days before the exam learning week 4 content (just to fail irl technical interviews)

Course forums / ronnie_low.low@unsw.edu.au

Introductions

- I'm Ronnie
- Your 2521 tutor this term!
- I also teach 3121! So might be seeing you around in the future
- I play guitar
- Diamond in brawlhalla (not anymore)
- div3 UGC in tf2 (peaked placements)
- Secretary of Linux Society

- Your turn! Introduce yourself to the people on your table
 - Discuss what you've learnt from COMP1511
 - Discuss any parts you're unsure of/things you don't remember

Revision

- Points to an address in memory
- Data is stored at a given address in memory
- A pointer is just a number that tells you where to look
- Kinda like a street address!
 - A street address doesn't tell you what colour a house is, but it gives you the information to go there and look
- Lets look at an example! (Diagram drawing)

- C handles stack memory for us!
- Stack memory is every piece of data that is created without malloc/calloc/realloc
- How does the stack work?
 - Diagram time!
- Stack memory gets deallocated (destroyed) when the stack frame stops existing
- Heap memory lives forever until we call `free()` or the program ends (memory leak!)

Examples

Stack vs Heap example

Explain how these two pieces of code differ:

```
int main(void) {  
    stackInt();  
}  
  
void stackInt(void) {  
    int a = 5;  
}
```

```
int main(void) {  
    heapInt();  
}  
  
void heapInt(void) {  
    int *a = malloc(sizeof(int));  
    *a = 5;  
}
```

Answer:

- The variable in `stackInt(void)` stops existing once the `stackInt()` function finishes executing, since it is within the `stackInt()` stack frame
- The variable in `heapInt()` continues existing even after `heapInt()` finishes running, since it is on the heap (called by `malloc`)

Examples

Swap example

Explain why the `swap()` function here does not work as intended:

```
int main(void) {  
    int a = 5;  
    int b = 7;  
    swap(a, b);  
    printf("a = %d, b = %d\n", a, b);  
}  
  
void swap(int a, int b) {  
    int tmp = a;  
    a = b;  
    b = tmp;  
}
```

Answer

- When we pass in variables as arguments to a function, the values are copied into the function stack.
- The change occurs on the function stack, which is local to the function only. When we conduct a swap, we're only swapping the copied values in our function! Our original values are never touched.
- Any suggestions on how to fix this?

Examples

Heap allocation example

Modify the code below so that it allocates the struct on the heap, instead of the stack.

```
struct node {  
    int value;  
    struct node *next;  
};
```

```
int main(void) {  
    struct node n;  
    n.value = 42;  
    n.next = NULL;  
}
```

Answer

```
struct node {  
    int value;  
    struct node *next;  
};  
  
int main(void) {  
    struct node *n = malloc(sizeof(struct node));  
    n->value = 42;  
    n->next = NULL;  
}
```

The following code creates an array of 5 integers on the stack and uses it to store some values. How can you allocate the array on the heap instead?

```
int main(void) {  
    int a[5];  
    for (int i = 0; i < 5; i++) {  
        a[i] = 42;  
    }  
}
```

Answer

```
int main(void) {  
    int *a = malloc(sizeof(int) * 5);  
    for (int i = 0; i < 5; i++) {  
        a[i] = 42;  
    }  
}
```

- Linked lists are a data structure allocated on the heap that stores data
- Linked lists comprise of nodes, where each node holds a piece of data
- Linked list items aren't contiguous in memory (they aren't next to each other)
- Each node of a linked list also holds a pointer to the next linked list node
- House analogy

```
struct node {  
    int data,  
    struct node *next  
};
```

Consider the following two linked list representations:

```
// Representation 1
struct node {
    int value;
    struct node *next;
};

int listLength(struct node *list);
```

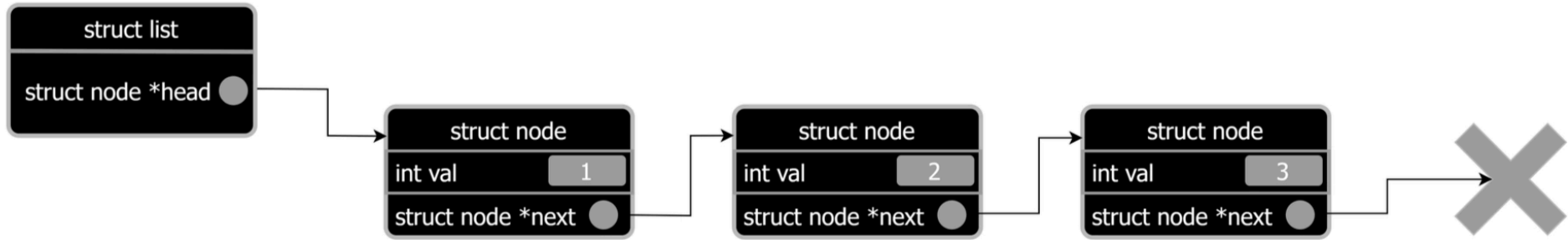
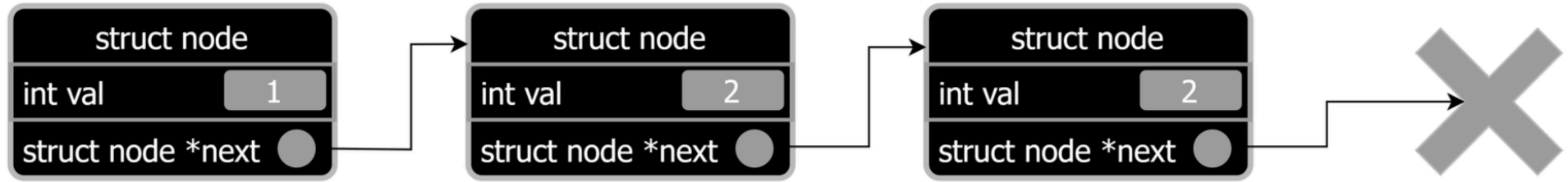
```
// Representation 2
struct node {
    int value;
    struct node *next;
};

struct list {
    struct node *head;
};

int listLength(struct list *list);
}
```

- Compare the two representations diagrammatically.

Answer:



TM ravindu herath

```
// Representation 1
struct node {
    int value;
    struct node *next;
};

int listLength(struct node *list);
```

```
// Representation 2
struct node {
    int value;
    struct node *next;
};

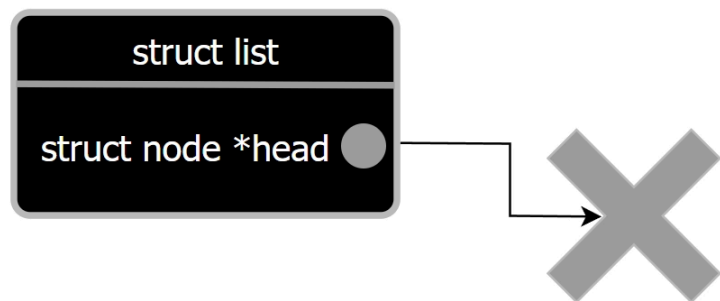
struct list {
    struct node *head;
};

int listLength(struct list *list);
}
```

- How is an empty list represented in each case?

Answer

- Representation 1: `struct node *list = NULL;`
- Representation 2:



```
// Representation 1
struct node {
    int value;
    struct node *next;
};

int listLength(struct node *list);
```

```
// Representation 2
struct node {
    int value;
    struct node *next;
};

struct list {
    struct node *head;
};

int listLength(struct list *list);
}
```

- What are the advantages of having a separate list struct as in Representation 2?

Answer

- A separate wrapper list struct can be very useful!
- A wrapper on a snack packet tells you the nutrition, calories, weight and other information about the snack without you measuring these values yourself.
- For instance, we can store information about our linked list, like the length of the list and update the length when we insert/remove a value from the linked list! That way, we can determine the length of our linked list without traversing and counting the size every time.

Write a function to sum the values in the list. Implement it first using `while` and then using `for` .

```
struct node {  
    int value;  
    struct node *next;  
};
```

Answer

```
int list_sum_while(struct node *head) {  
    struct node *curr = head;  
    int sum = 0;  
  
    while (curr != NULL) {  
        sum += curr->data;  
        curr = curr->next;  
    }  
  
    return sum;  
}
```

```
int list_sum_for(struct node *head) {  
    int sum = 0;  
  
    for (struct node *curr = head; curr != NULL; curr = curr->next) {  
        sum += curr->data;  
    }  
    return sum;  
}
```

Implement a function to delete the first instance of a value from a list, if it exists. The function should return a pointer to the start of the updated list. Use the following list representation and prototype:

```
struct node {  
    int value;  
    struct node *next;  
};  
  
struct node *listDelete(struct node *list, int value);
```

```
struct node *listDelete(struct node *list, int value) {
    // if list is empty
    if (list == NULL) {
        return NULL;
    }

    struct node *ret = list;
    struct node *curr = list;
    struct node *prev = curr;

    while (curr != NULL) {
        if (prev == curr && curr->value == value) {
            // the value we are trying to replace is the very first value
            ret = curr->next;
            free(curr);
            return ret;
        } else if (curr->value == value) {
            // not the first value
            prev->next = curr->next;
            free(curr);
        }
    }
}
```

```
        return ret;
    }
    prev = curr;
    curr = curr->next;
}
// we failed to find a value in the linked list that has value, so we delete nothing
return ret;
}
```

How would the implementation and prototype be different if the following list representation was used instead?

```
struct node {  
    int value;  
    struct node *next;  
};  
  
struct list {  
    struct node *head;  
};
```

Answer

- We no longer have to modify the value we're returning!
- We can simply just return the same head! Head's value needs to be changed to point to the “new” head of the linked list

```
void *listDelete(struct list *list, int value) {
    struct node *head = list->head;
    // if list is empty
    if (head == NULL) {
        return;
    }

    struct node *curr = head;
    struct node *prev = head;

    while (curr != NULL) {
        if (prev == curr && curr->value == value) {
            // the value we are trying to replace is the very first value
            list->head = curr->next;
            free(curr);
            return;
        } else if (curr->value == value) {
            // not the first value
            prev->next = curr->next;
            free(curr);
        }
    }
}
```

```
    return;
}
prev = curr;
curr = curr->next;
}
// we failed to find a value in the linked list that has value, so we delete nothing
return;
}
```